INFORME TÉCNICO: SISTEMA DE GESTIÓN DE TORNEO DE VIDEOJUEGOS

Ingeniería en Informática

Paradigmas de Programación II

Trabajo Práctico: TP 6 - Integrador

Integrantes:

- GARZÓN LOZA, Rocío Desiré
- SÁNCHEZ SOLANO, Juan Cruz
- DARRULL, Farid

Fecha de entrega: 13-4-2026

1. Introducción

En este informe se detalla el diseño de un sistema de programación en base a objeto desarrollado en Java, el cual se encargará de gestionar un torneo de videojuegos.

El prototipo permite realizar la inscripción de jugadores, simular emparejamientos aleatorios y registrar automáticamente los resultados obtenidos.

Para el desarrollo de este se aplicaron conceptos fundamentales tales como:

- Colecciones: Utilizados para el manejo de los participantes, partidas y competiciones.
- Encapsulamiento: Para asegurar la integridad de los objetos de los jugadores.
- Interacción entre objetos: Para modelar como las partidas afectan directamente las estadísticas de los competidores.

También se describen las decisiones de diseño, la estructura de clases definida y la representación de un escenario específico mediante diagrama de objetos.

2. Diseño (Explicación del Modelo)

Elegimos utilizar 4 clases para estructurar el sistema:

Esta decisión fue basada en la necesidad de representar diferentes videojuegos que generarían diferentes competencias, partidas dentro de las competencias, jugadores y un sistema que administre las anteriores mencionadas.

- **Sistema:** Como su nombre indica, maneja el sistema general del torneo, como la cantidad de rondas e inscripciones. Para esto, utiliza un **ArrayList** que permite gestionar la lista de jugadores de forma dinámica. También se encarga del inicio de la competición, la visualización del ranking, el historial de partidas y competencias y la búsqueda de jugadores por nombre. Además, integra la clase **Scanner** para la interacción por terminal.
- **Competición:** Se encarga de organizar el torneo mediante el uso de **colecciones** para agrupar las partidas. Su función principal es realizar los emparejamientos, ejecutarlos y denominar un ganador. Cada objeto de esta clase representa una competición vinculada a un juego distinto.
- **Partida:** Esta clase realiza el enfrentamiento entre dos instancias de la clase **Jugador**. El resultado se decide de manera **aleatoria** (simulando el azar del juego). Al finalizar, se encarga de sumar el punto correspondiente al ganador y actualizar las estadísticas de ambos participantes.
- **Jugador:** Contiene toda la información y atributos de cada competidor, como el nombre, DNI, puntaje acumulado y cantidad de partidas jugadas. También maneja sus estados internos mediante variables booleanas para saber si el jugador está **emparejado** o **eliminado** del torneo (competencia) actual.

Diagrama de Clases:

3. Escenario de Ejecución (Diagrama de Objetos)

El torneo tiene 3 jugadores: Ana, Luis y Pedro.

Se jugaron 2 partidas:
● Ana vs Luis (gana Ana)
● Pedro vs Ana (empate)

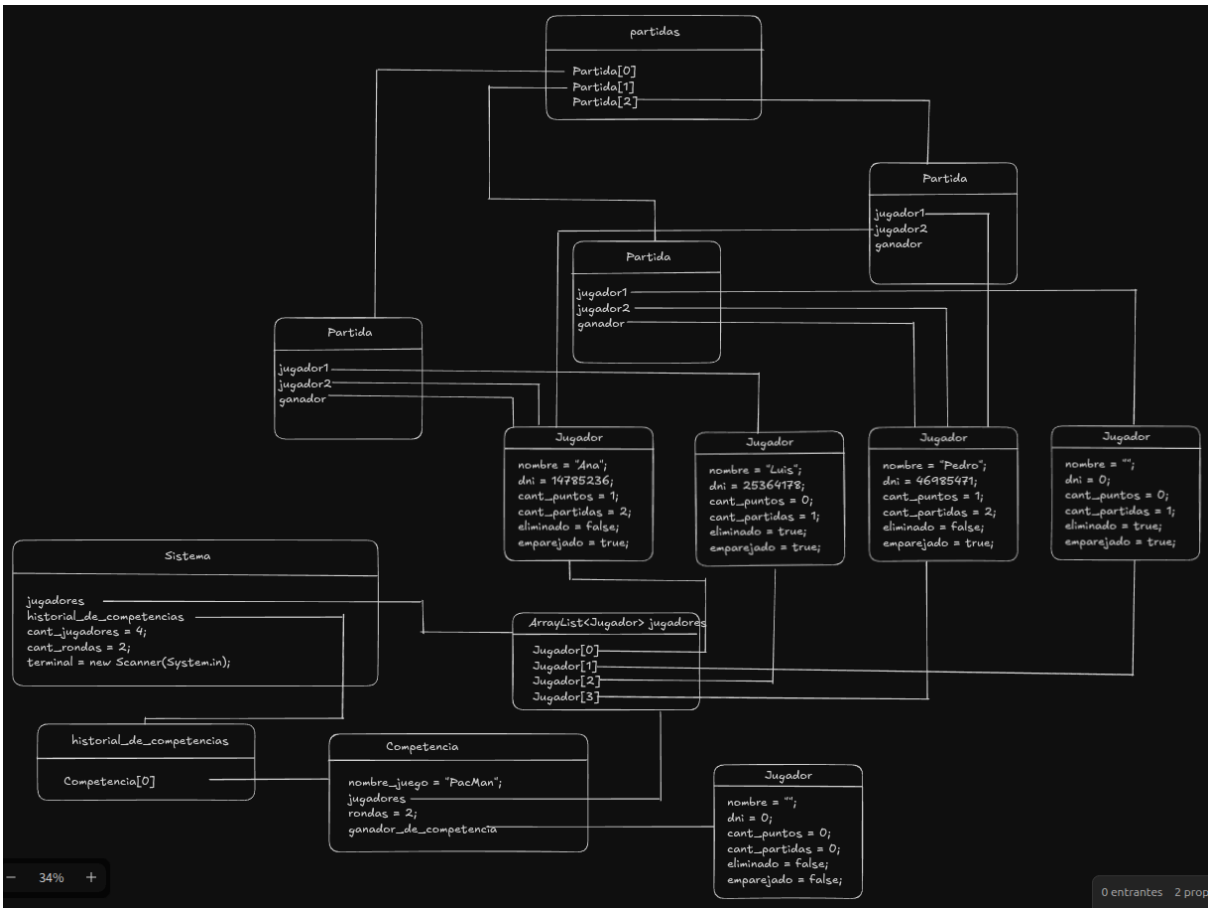

Diagrama de objetos simulación de competencia del juego “PacMan” con 3 jugadores. El sistema se encarga de la carga de jugadores e iniciar la competencia, luego se asignan las parejas para las partidas y los puntajes.
Tener en cuenta que el diagrama de objetos se ajusta a la consigna requerida, en una implementación del programa luego de la asignación del puntaje se determina un ganador de la competencia según la cantidad de puntos.

4. Detalles de Implementación

En esta sección se describen los aspectos técnicos más relevantes del código desarrollado en BlueJ.

4.1. Gestión de Colecciones e Iteración

Para la administración dinámica de los datos se optó por el uso de la clase de biblioteca ArrayList.

- En la clase Sistema, se utilizan “jugadores” e “historial_de_competencias” para almacenar las instancias creadas.
- Se implementan estructuras de control for y for-each para recorrer las listas en funcionalidades como la búsqueda de jugadores por nombre [buscarJugador()], la generación de rankings y la impresión de historiales.

- Para la función de mostrarRanking() se implementó un algoritmo de ordenamiento tipo burbuja que compara los puntajes de los objetos de tipo “Jugador” dentro de “jugadores” para mostrarlos de mayor a menor.

4.2. Lógica de Aleatoriedad y Simulación

Cumpliendo con la consigna de simular enfrentamientos al azar

- Se utilizó la clase java.util.Random dentro de competencia para seleccionar los índices aleatorios del Arraylist de jugadores y generar los emparejamientos.
- En la clase Partida, se utiliza Random para definir el resultado de cada enfrentamiento. Un valor aleatorio determina si gana el jugador1, jugador2 o si ocurre en empate(el cual se resuelve mediante el método casoEmpate()).

4.3. Encapsulamiento y Modificadores de Acceso

Se aplicó de manera estricta el concepto de ocultamiento de datos en la clase Jugador:

- Todos los atributos (nombre, dni, puntaje, etc.) fueron definidos como private.
- El acceso y la modificación de estos atributos se realiza exclusivamente a través de métodos Getters y Setters públicos.

4.4. Interacción entre Objetos y Manejo de Estados

- Competencia actúa como coordinador, creando objetos de tipo Partida;
- Las instancias de Partida reciben como parámetros objetos de tipo Jugador, modificando directamente sus estados internos [setEliminado()] y estadísticas [modificarPuntajes()] según el resultado de la simulación.
- Se utilizan banderas booleanas (emparejado, eliminado) para controlar el flujo de la competencia y asegurar que solo los jugadores aptos participen en cada ronda.

4.5. Validaciones y Control.

Se incluyeron controles para mejorar la experiencia de usuario y la integridad de los datos:

- Evitar duplicados: En la clase Sistema, el método comprobarRepetidos() recorre la lista de jugadores antes de confirmar una inscripción, comparando el DNI para evitar ingresos duplicados.
- Normalización de texto: Se utilizaron métodos como .toLowerCase y .trim() al buscar jugadores para que la comparación de nombres no sea sensible a espacios o mayúsculas.

5. Conclusiones

El desarrollo de este sistema permitió integrar con éxito los pilares de la Programación Orientada a Objetos en java. La estructura basada en clases con responsabilidades claras (Sistema, Competencia, Partida y Jugador) facilitó el manejo de la lógica del torneo y el seguimiento de estadísticas.
Gracias al uso de ArrayList, el sistema gestiona de forma dinámica a los competidores, mientras que el uso de encapsulamiento de los atributos asegura que los datos de los jugadores solo se modifiquen mediante reglas controladas. Finalmente, la implementación

de la clase Random permitió simular enfrentamientos aleatorios de manera eficiente, cumpliendo con todos los requerimientos técnicos y funcionales solicitados.